

Exception Handling

**Building Robust and Crash-Proof
Applications**

Learning Objectives

- **What We Will Learn**
 - Understand what an Exception is and why it occurs.
 - Differentiate between **Errors** and **Exceptions**.
 - Master the try, catch, finally blocks.
 - Distinguish between **Checked** and **Unchecked** exceptions.
 - Understand throw vs. throws.
 - Create Custom Exceptions.
 - **Special Focus:** Handling Exceptions in Swing GUIs.

What is an Exception?

- **Definition:** An event that disrupts the normal flow of program execution.
- **Cause:** Invalid user input, missing files, network issues, logic errors.
- **Goal:** Prevent the application from crashing (terminating abruptly).
- **Example:**
 - Dividing by zero.
 - Accessing an array index out of bounds.
 - Parsing a non-numeric String to an int

```
int x = 10 / 0; // ArithmeticException
int num = Integer.parseInt("ABC"); //
NumberFormatException
```

The Exception Hierarchy

- `java.lang.Throwable`: The root class.
 - Error: Serious system issues (e.g., `OutOfMemoryError`). Do not catch.
 - Exception: Conditions a program should catch.
 - Checked Exceptions: Checked at compile-time (e.g., `IOException`).
 - Unchecked Exceptions: Runtime errors (e.g., `NullPointerException`, `ArithmeticException`).

Try-Catch Block

- **Basic Handling: try and catch**
 - **try**: Block of code that might throw an exception.
 - **catch**: Block of code that handles the exception.

```
try {  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero!");  
}  
System.out.println("Program continues...");
```

- **Result:** The program does **not** crash; it prints the message and continues

Multiple Catch & Finally

- Advanced Flow Control
- **Multiple Catch Blocks:** Handle different exceptions differently.

```
try {  
    // risky code  
} catch (ArithmeticException e) {  
    // handle math error  
} catch (NumberFormatException e) {  
    // handle input error  
}
```

- **finally Block**
 - Executes **always** (whether exception occurs or not).
 - Used for cleanup (closing files, database connections).

```
finally {  
    System.out.println("Execution complete.");  
}
```

Throw vs. Throws

- **throw** (Keyword):
 - Used inside a method to explicitly throw an exception.
 - Followed by an object.

```
if (age < 18) throw new ArithmeticException("Not eligible");
```

- **throws** (Clause)
 - Used in **method signature** to declare potential exceptions.
 - Followed by a **class name**.

```
public void readFile() throws IOException { ... }
```

Checked vs. Unchecked Exceptions

- **Checked Exceptions:**

- Checked at **compile-time**.
- Must be handled (try-catch) or declared (throws).
- Examples: IOException, SQLException, ClassNotFoundException.

- **Unchecked Exceptions (Runtime):**

- Checked at **runtime**.
- Usually programming errors.
- Examples: NullPointerException, ArrayIndexOutOfBoundsException.

- **Rule:** You *must* handle Checked exceptions; you *should* fix Unchecked exceptions.

Creating Custom Exceptions

- **Why?** Standard exceptions don't always describe your specific business logic error.
- **How:** Extend Exception (Checked) or RuntimeException (Unchecked)

```
public class InvalidAgeException extends Exception {  
    public InvalidAgeException(String message) {  
        super(message) ;  
    }  
}
```

- **Usage**

```
if (age < 0) throw new InvalidAgeException("Age  
cannot be negative");
```

Exceptions in Swing GUIs

- **Handling Exceptions in GUI Apps**
 - **Problem:** Printing to console (System.out.println) is invisible to users.
 - **Solution:** Use JOptionPane to show error dialog

```
try {  
    int val = Integer.parseInt(textField.getText());  
} catch (NumberFormatException e) {  
    JOptionPane.showMessageDialog(frame,  
        "Please enter a valid number!",  
        "Input Error",  
        JOptionPane.ERROR_MESSAGE);  
}
```

- **Best Practice:** Never let a GUI crash silently. Always inform the user.

Best Practices

- Exception Handling Best Practices
 - ☒ Do:
 - Catch specific exceptions (not just Exception).
 - Log exceptions (use Logger, not just printStackTrace).
 - Clean up resources in finally or use Try-With-Resources.
 - Inform the user in GUIs.
 - ☒ Don't:
 - Swallow exceptions (empty catch block).
 - Use exceptions for normal flow control (e.g., don't use exception to break a loop).
 - Expose stack traces to end-users (security risk).

Summary & Lab

- **Summary:**
 - Use try-catch to prevent crashes.
 - Use finally for cleanup.
 - Handle Checked exceptions explicitly.
 - Show user-friendly dialogs in Swing.
- **Lab Exercise:**
 - Modify your Temperature Converter (from Swing module).
 - Add try-catch to handle non-numeric input.
 - Show a JOptionPane error if input is invalid.
 - Ensure the app does not crash on bad input.